1. Tp sur transaction spring et accès aux données (jdbc,jpa) sans spring-data

NB1 : cette série de Tps peut soit être effectuée pas à pas si l'on souhaite bien comprendre la structure des anciennes applications Spring (basées uniquement sur Spring-framework) .
Si par contre, on préfère se contenter d'un très rapide aperçu pour la compréhension , on pourra directement analyser et faire fonctionner la solution (projet "appliSpringSansSpringBoot" sans "debut….") .

NB2 : le contenu initial du projet "***debutAppliSpringSansSpringBoot***" du référentiel maven https://github.com/didier-tp/spring2026.git comporte une bonne partie des éléments des Tps ci-après → énoncés simplifiés (vérifier présence et contenu d'un fichier à améliorer plutôt que copier/coller) . En règle générale , le contenu initial des fichiers est incomplet (il faut ajouter des annotations et d'autres morceaux de code) .

1.1. Accès aux données (DataSource JDBC) , DAO

- Créer package ***tp.appliSpring.core.entity***
- Créer classe ***Compte.java***

```
package tp.appliSpring.core.entity;

public class Compte {

   private Long numero;
   private String label;
   private Double solde;

 //+get/set , constructeurs , toString()
}
```

- Créer package ***tp.appliSpring.core.dao***
- Créer interface ***DaoCompte.java***

```
package tp.appliSpring.core.dao;

import java.util.List;
import tp.appliSpring.core.entity.Compte;

public interface DaoCompte{
        Compte findById(Long numCpt);
        Compte save(Compte compte); //sauvegarde au sens saveOrUpdate
        List<Compte> findAll();
        void deleteById(Long numCpt);
   //...
}
```

- dans ***src/main/resources*** ajouter **application.properties** avec ce contenu :

```
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.url=jdbc:h2:~/mydbbank
spring.datasource.username=sa
spring.datasource.password=
```

- dans ***tp.appliSpring.core*** ajouter **MySpringApplication** avec ce contenu :

```
package tp.appliSpring.core;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

//version sans springBoot
@Configuration
@ComponentScan(basePackages = { "tp.appliSpring.core"})
//NB : Tous les sous packages de tp.appliSpring.core seront scrutés pour y découvrir
//@Component… et aussi pour y découvir d'autres classes avec @Configuration
public class MySpringApplication {

	public tatic void main(String[] args) {
		//System.setProperty("spring.profiles.active", "p1");

		AnnotationConfigApplicationContext springContext = new
				AnnotationConfigApplicationContext(MySpringApplication.class) ;
		//...
		springContext.close();
	}
}
```

- Créer package ***tp.appliSpring.core.config***
- Créer la classe ***CommonConfig.java***

```
package tp.appliSpring.core.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.context.support.PropertySourcesPlaceholderConfigurer;

@Configuration
@PropertySource("classpath:/application.properties")
public class CommonConfig {

	@Bean
	public static PropertySourcesPlaceholderConfigurer
	propertySourcesPlaceholderConfigurer(){
		return new PropertySourcesPlaceholderConfigurer();
		//pour pouvoir interpréter ${} in @Value()
	}
}
```

- Créer la classe ***DataSourceConfig.java***

```
package tp.appliSpring.core.config;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
public class DataSourceConfig {

        @Value("${spring.datasource.driverClassName}")
        private String jdbcDriver;

        @Value("${spring.datasource.url}")
        private String dbUrl;

        @Value("${spring.datasource.username}")
        private String dbUsername;

        @Value("${spring.datasource.password}")
        private String dbPassword;

   @Bean(name="dataSource")
   public DataSource dataSource() {
                DriverManagerDataSource dataSource = new DriverManagerDataSource();
                dataSource.setDriverClassName(jdbcDriver);
                dataSource.setUrl(dbUrl);
                dataSource.setUsername(dbUsername);
                dataSource.setPassword(dbPassword);
                return dataSource;
   }

  //seulement utile pour le dao en version Jdbc (avec NamedParameterJdbcTemplate):
  @Bean()
   public NamedParameterJdbcTemplate namedParameterJdbcTemplate( DataSource dataSource) {
      return new NamedParameterJdbcTemplate(dataSource);
   }

}
```

- dans ***src/test/java*** et dans un package ***tp.appliSpring.dao*** à créer, ajouter cette classe de test :

```
package tp.appliSpring.dao;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import tp.appliSpring.core.MySpringApplication;
import tp.appliSpring.core.dao.DaoCompte;
import tp.appliSpring.core.entity.Compte;

@ExtendWith(SpringExtension.class) //si junit5/jupiter
@ContextConfiguration(classes= {MySpringApplication.class})
public class TestCompteDao {

   private static Logger logger = LoggerFactory.getLogger(TestCompteDao.class);

        @Autowired
        @Qualifier("jdbc")
        //@Qualifier("jpa")
        private DaoCompte daoCompte; //à tester

 @Test
 public void testAjoutEtRelectureEtSuppression() {
        //hypothese : base avec tables vides et existantes au lancement du test
        Compte compte = new Compte(null,"compteA",100.0);
        Compte compteSauvegarde = this.daoCompte.save(compte); //INSERT INTO
        logger.debug("compteSauvegarde=" + compteSauvegarde);

        Compte compteRelu = this.daoCompte.findById(compteSauvegarde.getNumero());
        Assertions.assertEquals("compteA",compteRelu.getLabel());
        Assertions.assertEquals(100.0,compteRelu.getSolde());
        logger.debug("compteRelu apres insertion=" + compteRelu);

        compte.setSolde(150.0); compte.setLabel("compte_a");
        Compte compteMisAjour = this.daoCompte.save(compte); //UPDATE
        logger.debug("compteMisAjour=" + compteMisAjour);

        compteRelu = this.daoCompte.findById(compteSauvegarde.getNumero()); //SELECT
        Assertions.assertEquals("compte_a",compteRelu.getLabel());
        Assertions.assertEquals(150.0,compteRelu.getSolde());
        logger.debug("compteRelu apres miseAjour=" + compteRelu);

        //+supprimer :
        this.daoCompte.deleteById(compteSauvegarde.getNumero());

        //verifier bien supprimé (en tentant une relecture qui renvoi null)
        Compte compteReluApresSuppression =
                this.daoCompte.findById(compteSauvegarde.getNumero());
        Assertions.assertTrue(compteReluApresSuppression == null);
        }
}
```

Script de préparation de la base de données (ici en version H2) :

init_db.sql

```
DROP TABLE IF EXISTS Compte;

CREATE TABLE Compte(
	numero integer auto_increment NOT NULL,
	label VARCHAR(64),
	solde double,
	PRIMARY KEY(numero));

INSERT INTO Compte (label,solde) VALUES ('compte courant',100);
INSERT INTO Compte (label,solde) VALUES ('compte codevi',50);
INSERT INTO Compte (label,solde) VALUES ('compte 3',150);

SELECT * FROM Compte;
```

set_env.bat

```
set MVN_REPOSITORY=C:\Users\administrateur\.m2\repository
set MY_H2_DB_URL=jdbc:h2:~/mydbbank
set H2_VERSION=2.2.224
set H2_CLASSPATH=%MVN_REPOSITORY%\com\h2database\h2\%H2_VERSION%\h2-%H2_VERSION%.jar
```

create_h2_database.bat

```
cd /d %~dp0
call set_env.bat
java -classpath %H2_CLASSPATH% org.h2.tools.RunScript -url %MY_H2_DB_URL%  -user sa -script init_db.sql -showResults
pause
```

lancer_console_h2.bat

```
cd /d %~dp0
call set_env.bat
java -jar  %H2_CLASSPATH% -user "sa" -url %MY_H2_DB_URL%

REM NB: penser à se déconnecter pour éviter des futurs verrous/blocages
pause
```

NB : Toute cette structure de code et configuration sera utilisée dès le(s) TP(s) suivant(s)

1.2. Petit exemple de DAO via JDBCTemplate

NB: Ce TP pas fondamental est facultatif : à faire ou pas selon le temps disponible

coder le début de ***DaoCompteJdbc.java*** avec le code suivant (*à compléter*)

```
package tp.appliSpring.core.dao;
```

```
import java.sql.ResultSet;     import java.sql.SQLException;
import java.util.HashMap;    import java.util.List;    import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.jdbc.core.namedparam.SqlParameterSource;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.jdbc.support.KeyHolder;
import org.springframework.stereotype.Repository;
import tp.appliSpring.core.entity.Compte;

@Repository //@Component de type DAO/Repository
@Qualifier("jdbc")
public class DaoCompteJdbc /*extends JdbcDaoSupport*/ implements DaoCompte {

private final String INSERT_SQL = "INSERT INTO compte(label, solde)  values(:label,:solde)";
private final String UPDATE_SQL = "UPDATE compte set label=:label , solde=:solde where numero=:numero";
private final String FETCH_ALL_SQL = "select * from compte";
private final String FETCH_BY_NUM_SQL = "select * from compte where numero=:numero";
private final String DELETE_BY_NUM_SQL = "delete from compte where numero=:numero";

@Autowired
private NamedParameterJdbcTemplate namedParameterJdbcTemplate;

@Override
public Compte findById(Long numCpt) {
	Compte compte = null;
	Map<String, Long> parameters = new HashMap<String, Long>();
	parameters.put("numero", numCpt);
	List<Compte> comptes = namedParameterJdbcTemplate.query(FETCH_BY_NUM_SQL,
	                                                        parameters,  new CompteMapper());
	compte = comptes.isEmpty()?null:comptes.get(0);
	return compte;
}

@Override
public Compte save(Compte compte) {
	if(compte==null)
		throw new IllegalArgumentException("compte must be not null");
	return (compte.getNumero()==null)?insert(compte):update(compte);
}

public Compte insert(Compte compte) {
	KeyHolder holder = new GeneratedKeyHolder(); //to retreive auto_increment value of pk
	SqlParameterSource parameters = new MapSqlParameterSource()
	                                        .addValue("label", compte.getLabel())
	                                        .addValue("solde", compte.getSolde());
	namedParameterJdbcTemplate.update(INSERT_SQL, parameters, holder);
	compte.setNumero(holder.getKey().longValue());//store auto_increment pk in instance to return
	return compte;
}

public Compte update(Compte compte) {
	//A CODER/COMPLETER  EN TP
}
```

```
@Override
public List<Compte> findAll() {
        //A CODER/COMPLETER  EN TP
}

@Override
public void deleteById(Long numCpt) {
        //A CODER/COMPLETER  EN TP
}

}

//classe auxiliaire "CompteMapper" pour convertir Resultset jdbc en instance de la classe Compte :
class CompteMapper implements RowMapper<Compte> {
        @Override
        public Compte mapRow(ResultSet rs, int rowNum) throws SQLException {
                Compte compte = new Compte();
                compte.setNumero(rs.getLong("numero"));
                compte.setLabel(rs.getString("label"));
                compte.setSolde(rs.getDouble("solde"));
                return compte;
        }
}
```

- Compléter le code manquant de cette classe
- tester via le lancement de ***TestCompteDao*** (dans src/test/java)

1.3. Accès aux données via JPA/Hibernate

Ajouter dans le package ***tp.appliSpring.core.config*** la classe de configuration ***DomainAndPersistenceConfig.java*** suivante :

```
package tp.appliSpring.core.config;

import java.util.Properties;     import javax.sql.DataSource;
import jakarta.persistence.EntityManagerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.JpaVendorAdapter;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.orm.jpa.vendor.Database;
import org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableTransactionManagement() // "transactionManager" (not "txManager") is expected !!!
@ComponentScan(basePackages = { "tp.appliSpring.core.dao" ,
                                "tp.appliSpring.core.service" ,  "tp.appliSpring.core.init"})
public class DomainAndPersistenceConfig {

// JpaVendorAdapter (Hibernate ou OpenJPA ou ...)
```

```
@Bean
public JpaVendorAdapter jpaVendorAdapter() {
        HibernateJpaVendorAdapter hibernateJpaVendorAdapter =
                    new HibernateJpaVendorAdapter();
        hibernateJpaVendorAdapter.setShowSql(false);
        hibernateJpaVendorAdapter.setGenerateDdl(false);
        //hibernateJpaVendorAdapter.setDatabase(Database.MYSQL);
        hibernateJpaVendorAdapter.setDatabase(Database.H2);
        return hibernateJpaVendorAdapter;
}

// EntityManagerFactory
@Bean(name = { "entityManagerFactory" })
public EntityManagerFactory entityManagerFactory(JpaVendorAdapter jpaVendorAdapter,
                                                        DataSource dataSource) {
        LocalContainerEntityManagerFactoryBean factory =
                        new LocalContainerEntityManagerFactoryBean();
        factory.setJpaVendorAdapter(jpaVendorAdapter);
        factory.setPackagesToScan("tp.appliSpring.core.entity");
        factory.setDataSource(dataSource);

        Properties jpaProperties = new Properties(); // java.util
        jpaProperties.setProperty("javax.persistence.schema-generation.database.action",
                                "drop-and-create"); //JPA>=2.1
        factory.setJpaProperties(jpaProperties);
        factory.afterPropertiesSet();
        return factory.getObject();
}

// Transaction Manager for JPA or ...
@Bean(name = "transactionManager")
public PlatformTransactionManager transactionManager(
                        EntityManagerFactory entityManagerFactory) {
        JpaTransactionManager txManager = new JpaTransactionManager();
        txManager.setEntityManagerFactory(entityManagerFactory);
        return txManager;
}
}
```

Coder au sein du package ***tp.appliSpring.core.dao*** la classe ***DaoCompteJpa***
en partant du code suivant (à compléter) :

```
package tp.appliSpring.core.dao;

import java.util.List;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
import tp.appliSpring.core.entity.Compte;

@Repository //@Component de type DAO/Repository
```

```
@Qualifier("jpa")
public class DaoCompteJpa implements DaoCompte {

	@PersistenceContext
	private EntityManager entityManager;

	@Override
	public Compte findById(Long numCpt) {
		// A CODER/COMPLETER EN TP
	}

	/*
	public Compte save(Compte compte) {
		try {
			entityManager.getTransaction().begin();
			if(compte.getNumero()==null)
				entityManager.persist(compte);//INSERT INTO
			else
				entityManager.merge(compte);//UPDATE
			entityManager.getTransaction().commit();
		} catch (Exception e) {
			entityManager.getTransaction().rollback();
			e.printStackTrace();
		}
		return compte; //avec numero plus null (auto_incrémenté)
	}
	*/

	@Override
	@Transactional
	public Compte save(Compte compte) {
			if(compte.getNumero()==null)
				entityManager.persist(compte);//INSERT INTO
			else
				entityManager.merge(compte);//UPDATE
		return compte; //avec numero plus null (auto_incrémenté)
	}

	@Override
	public List<Compte> findAll() {
		return entityManager.createQuery("SELECT c FROM Compte c",
								Compte.class)
						.getResultList();
	}

	@Override
	@Transactional
	public void deleteById(Long numCpt) {
		// A CODER/COMPLETER EN TP
		Compte compte = ……................
		entityManager…….(compte);
	}
```

```
}
```

- Compléter le code de la classe ci-dessus
- Ajouter toutes les **annotations** manquantes et nécessaires dans la classe ***tp.appliSpring.core.entity.Compte*** (@Entity, @Id , ….., @GeneratedValue(strategy = GenerationType.IDENTITY)
- switcher de qualificatif ~~@Qualifier("jdbc")~~ **Qualifier("jpa")** au sein de la classe ***TestCompteDao***
- Lancer le test et corriger les éventuels problèmes/erreurs .

1.4. Service Spring et gestion des transactions

Créer le nouveau package **tp.appliSpring.core.service**
Ajouter y l'interface ***ServiceCompte*** suivante :

```
package tp.appliSpring.core.service;

import java.util.List;
import tp.appliSpring.core.entity.Compte;

public interface ServiceCompte {
    Compte rechercherCompteParNumero(long numero);
    List<Compte> rechercherTousComptes();
    List<Compte> rechercherComptesDuClient(long numClient);
    Compte sauvegarderCompte(Compte compte);
    void supprimerCompte(long numCpt);
    void transferer(double montant,long numCptDeb,long numCptCred);
}
```

Ajouter la classe d'implémentation ***ServiceCompteImpl*** suivante (*à compléter*) :

```
package tp.appliSpring.core.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import tp.appliSpring.core.dao.DaoCompte;
import tp.appliSpring.core.entity.Compte;

@Service //classe de Service prise en charge par spring
public class ServiceCompteImpl implements ServiceCompte{

@Qualifier("jpa")
@Autowired
private DaoCompte daoCompte=null;

public Compte rechercherCompteParNumero(long numero) {
        return daoCompte.findById(numero);
}
```

```
public Compte sauvegarderCompte(Compte compte) {
        return daoCompte.save(compte);
}

public List<Compte> rechercherTousComptes() {
        // A CODER/COMPLETER EN TP
}

public List<Compte> rechercherComptesDuClient(long numClient) {
        //return null; //version zero
        return this.rechercherTousComptes(); //V1 (provisoire)
        //future version V2 (via un nouvel appel sur DAO exploitant @ManyToOne ou bien ...)
}

public void supprimerCompte(long numCpt) {
        // A CODER/COMPLETER EN TP
}

@Transactional(/*propagation = Propagation.REQUIRED*/) //REQUIRED par defaut
public void transferer(double montant, long numCptDeb, long numCptCred) {
  try {
        // transaction globale initialisée dès le début de l'exécution de transferer
        Compte cptDeb = this.daoCompte.findById(numCptDeb);
              //le dao exécute son code dans la grande transaction
              //commencée par le service sans la fermer et l'objet cptDeb remonte à l'état persistant
        cptDeb.setSolde(cptDeb.getSolde() - montant);
        //this.daoCompte.save(cptDeb); //facultatif si @Transactional


        //idem pour compte à créditer
        Compte cptCred= this.daoCompte.findById(numCptCred);
        cptCred.setSolde(cptCred.getSolde() + montant);
        //this.daoCompte.save(cptCred) //facultatif si @Transactional

        //en fin de transaction réussie (sans exception) , toutes les modification effectuées
        //sur les objets à l'état persistant seront répercutées en base (.save() automatiques)
        } catch (Exception e) {
                throw new RuntimeException("echec virement " + e.getMessage() , e);
                 //rollback se fait de façon fiable
                //ou bien throw new
               //ClasseExceptionPersonnaliseeHeritantDeRuntimeException("echec virement" , e);
        }
}
}
```

Au sein de ***src/test/java*** et du package ***tp.appliSpring.core.service*** (à créer) , ajouter la classe de test ***TestServiceCompte*** suivante :

```
package tp.appliSpring.service;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import tp.appliSpring.core.MySpringApplication;
import tp.appliSpring.core.entity.Compte;
import tp.appliSpring.core.service.ServiceCompte;

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes= {MySpringApplication.class})
//@ActiveProfiles({ "embeddedDB" , "dev" , "perf" })
public class TestServiceCompte {

private static Logger logger = LoggerFactory.getLogger(TestServiceCompte.class);

@Autowired
private ServiceCompte serviceCompte; //à tester


@Test
public void testVirement() {
        Compte compteASauvegarde = this.serviceCompte.sauvegarderCompte(
                new Compte(null,"compteA",300.0));
        Compte compteBSauvegarde = this.serviceCompte.sauvegarderCompte(
                 new Compte(null,"compteB",100.0));
        long numCptA = compteASauvegarde.getNumero();
        long numCptB = compteBSauvegarde.getNumero();
        //remonter en memoire les anciens soldes des compte A et B avant virement
        //(+affichage console ou logger) :
        double soldeA_avant= compteASauvegarde.getSolde();
        double soldeB_avant = compteBSauvegarde.getSolde();
        logger.debug("avant bon virement, soldeA_avant="+soldeA_avant +
                        " et soldeB_avant=" + soldeB_avant);

        //effectuer un virement de 50 euros d'un compte A vers vers compte B
        this.serviceCompte.transferer(50.0, numCptA, numCptB);

        //remonter en memoire les nouveaux soldes des compte A et B apres virement
        // (+affichage console ou logger)
        Compte compteAReluApresVirement =
                        this.serviceCompte.rechercherCompteParNumero(numCptA);
        Compte compteBReluApresVirement =
                       this.serviceCompte.rechercherCompteParNumero(numCptB);
        double soldeA_apres = compteAReluApresVirement.getSolde();
        double soldeB_apres = compteBReluApresVirement.getSolde();
        logger.debug("apres bon virement, soldeA_apres="+soldeA_apres
                         + " et soldeB_apres=" + soldeB_apres);

        //verifier -50 et +50 sur les différences de soldes sur A et B :
        Assertions.assertEquals(soldeA_avant - 50, soldeA_apres,0.000001);
```

```
        Assertions.assertEquals(soldeB_avant + 50, soldeB_apres,0.000001);
}

//@Test
public void testMauvaisVirement() {
        /*   VARIANTE A CODER/COLPLETER EN TP
        COPIER/COLLER à ADPATER de  testVirement()
         AVEC
        try {
            this.serviceCompte.transferer(50.0, numCptA, -numCptB); //erreur volontaire
        } catch (Exception e) {
                logger.error("echec normal du virement " + e.getMessage());
        }
        et
        //verifier -0 et +0 sur les différences de soldes sur A et B
        Assertions.assertEquals(soldeA_avant , soldeA_apres,0.000001);
        Assertions.assertEquals(soldeB_avant , soldeB_apres,0.000001);
         */
}
}
```

Série de tests à effectuer :

1. enlever ~~@Transactional~~ au dessus de la méthode **transferer** et enlever les commentaires sur les lignes *this.daoCompte.save(cptDeb);* *et this.daoCompte.save(cptCred);*
2. *lancer le test* ***testVirement()*** et corriger les bugs si nécessaire
3. coder et lancer ***testMauvaisVirement***() . c'est normal si ça ne fonctionne pas bien sans l'ajout de @Transactional
4. replacer @Transactional au dessus de la méthode **transferer** et relancer le test ***testMauvaisVirement***() qui devrait normalement fonctionner .
5. replacer des commentaires sur les lignes *this.daoCompte.save(cptDeb);* *et this.daoCompte.save(cptCred);*
   Tous les tests devraient encore bien fonctionner .